



Read Chapters 2, 4 and 5

Unit 1_04

Java Fundamentals

Loops

Outline

- Increment/decrement operators
- while loop
- do-while loop
- for loop
- break statement
- continue statement

Java Loops



The Increment and Decrement Operators

- In programming (Java, C, C++, JavaScript etc.), the **increment operator ++** increases the value of a variable by 1. Similarly, the **decrement operator --** decreases the value of a variable by 1.

```
int a=10, b = 5 , c = 6;
a++;
b--;
c++;
System.out.printf("a=%d , b=%d , c=%d %n" , a, b , c);
//Output => a=11 , b=4 , c=7
```

The Increment and Decrement Operators

++ and -- operator as prefix and postfix

- If you use the ++ operator as prefix like: **++var.** The value of var is **first** incremented by 1 **then**, it **returns the value.**
- If you use the ++ operator as postfix like: var++. The original value of var is **returned first** then, **var is incremented by 1.**
- The -- operator works in a similar way like the ++ operator except it decreases the value by 1.

```
int a=10, b = 5 , c = 6 , d , e;
d=a++; // d=a first (10) then a is incremented by 1 a = 11
c=--b; // b is decremented first then c=b (both = 5)
b=++a; // a is incremented first then b=a (both = 12)
e = --b + a-- ; //b is decremented first (11) then e = 11 + 12 then a is decremented
System.out.printf("a=%d , b=%d , c=%d , d=%d  e=%d %n" , a, b , c , d , e);
//Output a=11 , b=11 , c=4 , d=10  e=23
```

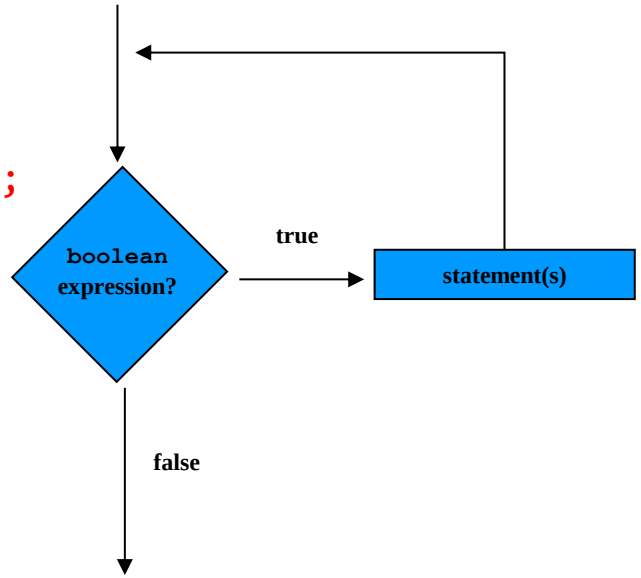
The Increment and Decrement Operators

Operator	Operator name	Sample expression	Explanation
`++`	prefix increment	`++a`	Increment `a` by `1`, then use the new value of `a` in the expression in which `a` resides.
`++`	postfix increment	`a++`	Use the current value of `a` in the expression in which `a` resides, then increment `a` by `1`.
`--`	prefix decrement	`--b`	Decrement `b` by `1`, then use the new value of `b` in the expression in which `b` resides.
`--`	postfix decrement	`b--`	Use the current value of `b` in the expression in which `b` resides, then decrement `b` by `1`.

The while Loop

- While the condition is **true**, the statements will **execute repeatedly**.
- The while loop is a **pretest** loop, which means that it will test the value of the condition prior to executing the loop.

```
while(condition)
{
        statements;
}
```




```
int i = 1;
while ( i <= 5) { // condition
System.out.println(i);
i++; // increment
}
```

```
1
2
3
4
5
```

Infinite Loop

- In order for a while loop to end, the **condition** must become **false**. The following loop will never end:

```
int i = 1;
while ( i <= 5) { // condition will never become false 1 =1
System.out.println(i);
}
```

```
1
1
1
1
... forever
```

- A while loop executes **0 or more** times. If the **initial condition is false**, the loop will not execute.

```
int i = 10;
while ( i <= 5) { // initial condition is false. Loop body skipped
System.out.println(i);
} // No output. Loop never executed
```

The while Loop for Input Validation

- Input validation is the process of ensuring that user input is valid.

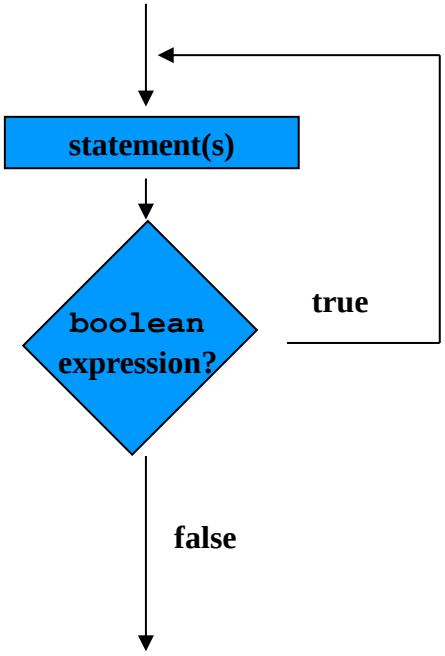
```
double gpa;
Scanner input = new Scanner(System.in);
System.out.print("Enter GPA (0-4.0): ");
gpa = input.nextDouble();
while ( gpa > 4 || gpa < 0) {//Validate the input
          System.out.print("Invalid GPA, Please re-enter a valid value: ");
          gpa = input.nextDouble();
// Keep getting until a valid value is entered
}
System.out.printf("Your GPA = %.2f %n", gpa);
```

```
Enter GPA (0-4.0): 99
Invalid GPA, Please re-enter a valid value: -8
Invalid GPA, Please re-enter a valid value: 22
Invalid GPA, Please re-enter a valid value: 3.7
Your GPA = 3.70
```

The do-while Loop

- The do-while loop is a **post-test** loop, which means it will execute the loop **prior** to testing the condition.

```
do
{
    statement(s);
}while (condition);
```




```
int i =1;
do {
System.out.println(i);
i++;
}while (i <= 3);
```

```
1
2
3
```

The do-while Loop

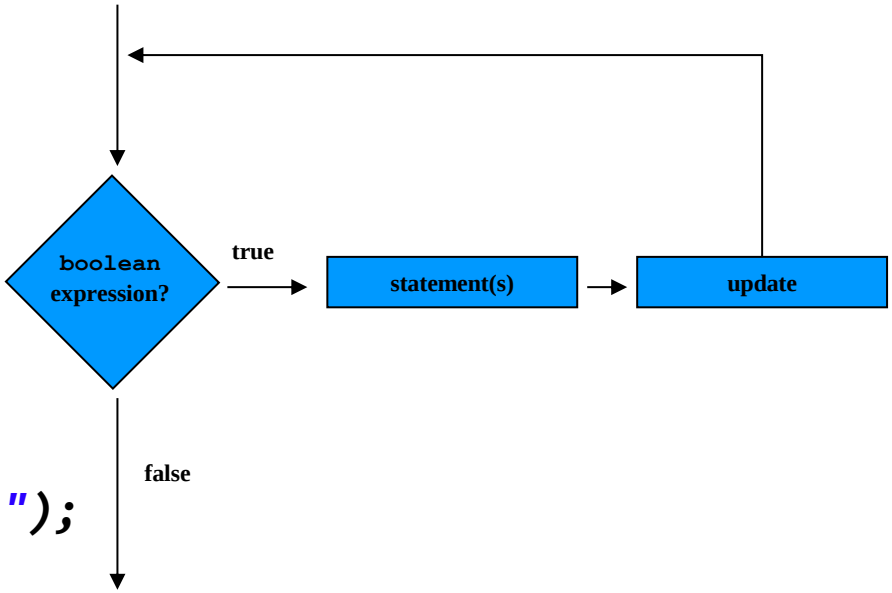
- The do-while loop executes **at least once** since the condition is tested at the **bottom** of the loop.

```
int i =99;
do { //Loop executed once
        System.out.println(i);
        i++;
}while (i <= 3);  //Condition is found false here
//Outputs 99 once
```

The for Loop

- A **pre-test** loop. The for loop allows the programmer to **initialize** a **control variable**, **test a condition**, and **modify the control variable** all in one line of code.

```
for(initialization; test; update)
{
statement(s);
}
```




```
int i; // Loop control variable
for (i = 1; i <= 5; i++)
 {
        System.out.print(i +" ");
 }
```

```
1 2 3 4 5
```

The for Loop

```
int number; // Loop control variable
 System.out.println("Number   Number Squared");
 System.out.println("----------------------");
 for (number = 1; number <= 5; number++)
 {
        System.out.println(number + "\t\t"
                          +number * number);
 }
```

```
Number   Number Squared
----------------------
1                1
2                4
3                9
4                16
5                25
```

```
int number;    // Loop control variable
int maxValue;  // Maximum value to display
System.out.println("I will display a table of " +
                          "numbers and their squares.");
Scanner keyboard = new Scanner(System.in);
System.out.print("How high should I go? ");
maxValue = keyboard.nextInt();
System.out.println("Number   Number Squared");
System.out.println("----------------------");
for (number = 1; number <= maxValue; number++)
   {
     System.out.println(number + "\t\t" + number * number);
   }
```

```
I will display a table of numbers and their squares.
How high should I go? 5
Number   Number Squared
----------------------
1                1
2                4
3                9
4                16
5                25
```

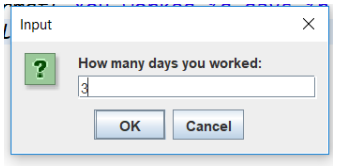
The for Loop : Total Sales

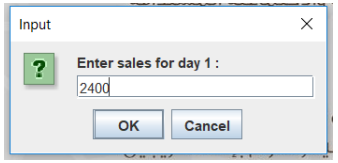
```java
int days;
double daySale, totalSales=0;
Scanner keyboard = new Scanner(System.in);
System.out.print("How many days you worked: ");
days = keyboard.nextInt();
for (int i=1; i <=days ;i++) {
        System.out.printf("Enter sales for day %d :" , i);
        daySale = keyboard.nextDouble();
        totalSales +=daySale;
        }
        System.out.printf("You worked %d days.%n" , days );
        System.out.printf("Total Sales: %.2f " , totalSales );
        keyboard.close();
```

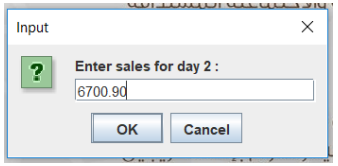
```
How many days you worked: 4
Enter sales for day 1 :1290.7
Enter sales for day 2 :3200.9
Enter sales for day 3 :1100.78
Enter sales for day 4 :2500.8
You worked 4 days.
Total Sales: 8093.18
```

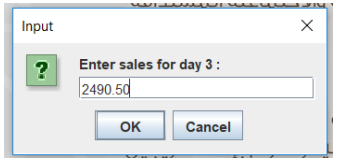
The for Loop : Total Sales with Dialog Boxes

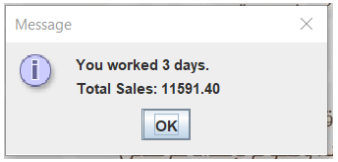
```
int days;
double daySale, totalSales=0;
String inputString;
inputString = JOptionPane.showInputDialog
            ("How many days you worked: ");
days = Integer.parseInt(inputString);
for (int i=1; i <=days ;i++) {
            inputString = JOptionPane.showInputDialog
            ("Enter sales for day " + i +" : ");
            daySale = Double.parseDouble(inputString);
            totalSales +=daySale;
}
String message = String.format("You worked %d days.%nTotal Sales: %.2f “
            , days,totalSales );
JOptionPane.showMessageDialog(null,message);
System.exit(0);
```











Sentinel Values

- Sometimes the end point of input data is **not known.**
- A **sentinel value** can be used to notify the program to **stop acquiring input.**
- The user could be prompted to input data that is not normally in the input data range (i.e. **–1** where normal input would be positive.)

```
int value, sum=0, count=0;
Scanner keyboard = new Scanner(System.in);
System.out.print("Enter an integer (-ve to exit): ");
value = keyboard.nextInt();
while ( value >= 0) {
  sum += value;
  count++;
  System.out.print("Enter an integer (-ve to exit): ");
  value = keyboard.nextInt();
}
System.out.printf("You entered %d values.%n" , count );
System.out.printf("Their sum = %d " , sum );
keyboard.close();
```

```
Enter an integer (-ve to exit): 133
Enter an integer (-ve to exit): 249
Enter an integer (-ve to exit): 900
Enter an integer (-ve to exit): -1
You entered 3 values.
Their sum = 1282
```

```
Enter an integer (-ve to exit): -1
You entered 0 values.
Their sum = 0
```

Sentinel Values : Total Sales

```
int days=0;
double daySale, totalSales=0;
Scanner keyboard = new Scanner(System.in);
System.out.print("Enter daily sales (-1 when finished): ");
daySale = keyboard.nextDouble();
// Accumulate the sales until -1 is entered.
while (daySale != -1)
{
  days++;// Increment days
  totalSales += daySale; //Accumulate sales
  // Get the next daily sales .
  System.out.print("Enter daily sales (-1 when finished):  ");
  daySale = keyboard.nextDouble();
}
 // Display the total Sales and number of days
System.out.printf("You worked %d days.%n" , days );
System.out.printf("Total Sales: %.2f " , totalSales );
keyboard.close();
```

```
Enter daily sales (-1 when finished): 1200
Enter daily sales (-1 when finished):  3450
Enter daily sales (-1 when finished):  12009
Enter daily sales (-1 when finished):  786.65
Enter daily sales (-1 when finished):  -1
You worked 4 days.
Total Sales: 17445.65
```

```
Enter daily sales (-1 when finished): 2399
Enter daily sales (-1 when finished):  98765
Enter daily sales (-1 when finished):  12334
Enter daily sales (-1 when finished):  12345
Enter daily sales (-1 when finished):  9087
Enter daily sales (-1 when finished):  6650
Enter daily sales (-1 when finished):  7800
Enter daily sales (-1 when finished):  9000
Enter daily sales (-1 when finished):  -1
You worked 8 days.
Total Sales: 158380.00
```

Nested Loops

- Like **if** statements, **loops** can be nested.
- If a loop is nested, the **inner loop will execute all of its iterations** for **each time the outer loop executes once.** The loop statements in this example will execute **100** times.

```
for(int i = 0; i < 10; i++)
    for(int j = 0; j < 10; j++)
        loop statements;
```

```
for (int i=1; i <3 ; i++)
{
        System.out.println("I ="+ i);
        for (int k=1; k<=5; k++)
                System.out.println("\tK=" + k);
}
```

```
I =1
        K=1
        K=2
        K=3
        K=4
        K=5
I =2
        K=1
        K=2
        K=3
        K=4
        K=5
```

The break Statement

- The **break** statement can be used to **abnormally terminate a loop.**
- The use of the break statement in loops bypasses the normal mechanisms and **makes the code hard to read** and maintain.
- It is considered bad form to use the break statement in this manner.

```
int k=1;
while ( k < 10) {
        System.out.println("K = " + k);
        k++;
        if ( k % 4 == 0)
                break;
}
System.out.println("I am out of the loop!");
```

```
K = 1
K = 2
K = 3
I am out of the loop!
```

The continue Statement

- The **continue** statement will cause the **currently** executing iteration of a loop to **terminate** and the **next** iteration will begin.
- The continue statement will cause the evaluation of the condition in while and for loops.
- Like the break statement, the continue statement should be avoided because it makes the code hard to read and debug.

```
for (int i=1; i <= 10 ; i++)
{
        if (i % 2 ==1)
                continue;
        System.out.println("I ="+ i);

}
```

```
I =2
I =4
I =6
I =8
I =10
```